

Dynamic Routing and Test-Time Fisher Information for Robust Riemannian Model Merging

Anonymous Authors¹

Abstract

Test-Time Model Merging (TTMM) is an emerging paradigm that dynamically fuses multiple task-specific expert neural networks into a single multi-task model at deployment time, adapting to a non-stationary stream of test data. Existing TTMM methods face two fundamental challenges. First, we identify a critical, previously unnoticed implementational bottleneck in existing TTMM pipelines: the complete omission of Batch Normalization (BN) running statistics during weight fusion, which we show severely degrades performance across all baselines by creating an activation mismatch. Second, when experts share a classification head (e.g., in multi-task classification), standard unsupervised entropy minimization suffers from the “feedback trap” (entropy collapse), where the model’s layer-wise merging coefficients collapse towards a single overconfident out-of-distribution expert that predicts a constant class. To resolve both limitations, we propose DR-Fisher (Dynamic Routing with Test-Time Fisher Riemannian Fine-tuning), a fully unsupervised, data-free, and robust TTMM framework. DR-Fisher first introduces proper Batch Normalization buffer merging with autograd-detached coefficient weights to preserve activation integrity. It then leverages a fully training-free Entropy-Based Expert Routing (EBER) mechanism to dynamically route each incoming test batch to its active expert on-the-fly, completely avoiding the feedback trap. Finally, EBER-routed coefficients are fine-tuned using a Riemannian gradient step pre-

conditioned by Test-Time Fisher Information (TT-Fisher) sensitivities computed dynamically on-the-fly on unlabeled test samples. Our extensive empirical evaluations on non-stationary vision streams under various environmental shifts (Gaussian noise, contrast shifts) demonstrate that DR-Fisher completely eliminates the dependency on source calibration data, achieves perfect routing, and delivers monumental performance gains, outperforming previous baselines by up to 58% absolute accuracy.

1. Introduction

The rapid proliferation of task-specific deep neural networks (referred to as “experts”) has led to a growing interest in model merging, where multiple fine-tuned models are combined into a single unified network without expensive retraining (Ainsworth et al., 2023; Wortsman et al., 2022; Matena & Raffel, 2022). A particularly challenging and realistic setting is Test-Time Model Merging (TTMM), where multiple experts must be merged dynamically at deployment to handle a non-stationary stream of test data from various domains or tasks (Yang et al., 2024; Zhao et al., 2024). In this setting, the mixing coefficients of the experts are adapted on-the-fly using unsupervised test-time objectives, such as entropy minimization (Wang et al., 2021).

Recent breakthroughs in TTMM, such as Fisher-Preconditioned Contrastive Alignment (FP-CA) and Information-Geometric Gradient Surgery (IGGS-Merge), highlight a crucial finding: optimizing merging coefficients uniformly across all layers leads to catastrophic representational collapse. Because neural network layers exhibit highly heterogeneous sensitivities, minor updates to early feature extractors or final classification heads can destroy the model’s structural integrity, while intermediate robust layers adapt too slowly. To address this, these frameworks pre-compute parameter-level diagonal Fisher Informa-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

tion on clean, labeled source calibration datasets to damp coefficient learning rates in sensitive layers.

Despite their success, existing TTMM methods suffer from three critical, unresolved limitations:

1. **Inaccessibility of Source Calibration Data:** In many real-world scenarios, the expert models’ training data is completely inaccessible at deployment time due to intellectual property, privacy regulations (e.g., medical data, personal user information), or storage limitations. A fully data-free TTMM method is highly desirable but currently lacking.
2. **Omission of Batch Normalization Statistics:** Existing TTMM implementations focus solely on merging the learnable weights and biases, completely omitting the Batch Normalization (BN) running buffers (running mean and variance). This forces the merged model to use pre-trained ImageNet running statistics or default values, creating a severe activation mismatch and limiting baseline performance.
3. **The Feedback Trap under Shared Output Heads:** When expert models share a classification head (e.g., when classifying into overlapping 10-way output classes), standard unconstrained entropy minimization fails. Because deep models are notoriously overconfident on out-of-distribution (OOD) data, minimizing prediction entropy collapses the layer-wise coefficients toward a single OOD expert. The model learns to predict a constant class with zero entropy but random classification accuracy (often 10%), a failure mode we term the “feedback trap.”

To overcome these boundaries, we introduce DR-Fisher (Dynamic Routing with Test-Time Fisher Riemannian Fine-tuning), a fully unsupervised, data-free, and robust parameter-sensitivity framework for test-time model merging. DR-Fisher introduces proper Batch Normalization buffer merging with autograd-detached coefficient weights, preserving activation statistics and restoring baseline feature representational quality.

To resolve the feedback trap, DR-Fisher leverages a fully training-free, unsupervised, and data-free Entropy-Based Expert Routing (EBER) mechanism. EBER passes each incoming unlabeled test batch through the separate expert networks, computes their respective predictive entropies, and identifies the active expert on-the-fly. We then initialize the layer-wise

merging coefficients close to the routed expert and fine-tune them using a Riemannian gradient step. This fine-tuning step is preconditioned by Test-Time Fisher Information (TT-Fisher), which estimates diagonal sensitivities dynamically and on-the-fly on unlabeled test samples using the model’s own pseudo-labels.

We evaluate DR-Fisher on a multi-task vision stream benchmark utilizing experts fine-tuned on MNIST (LeCun et al., 1998), FashionMNIST (Xiao et al., 2017), and KMNIST (Clanuwat et al., 2018) under rapid (Alternating) and gradual (Sequential) stream non-stationarity and multiple corruptions (Gaussian noise, contrast shift). Our results demonstrate that DR-Fisher completely avoids the feedback trap, achieves perfect routing, and delivers monumental performance improvements (over 58% absolute accuracy gains) over standard Euclidean or Fisher-based test-time adaptation.

2. Related Work

2.1. Model Merging

Model merging aims to combine multiple neural networks trained on different tasks or domains into a single unified model without the cost of training from scratch or joint multi-task training (McMahan et al., 2017; Ainsworth et al., 2023). Early approaches focused on simple coordinate or permutation alignment to resolve alignment issues prior to averaging (Jordan et al., 2022; Stoica et al., 2024). Modern multi-task merging methods such as Task Arithmetic (?), Ties-Merging (Yadav et al., 2023), and DARE (Yu et al., 2023) manage parameter interference by sparsifying, signing, and scaling weight deltas. RegMean (Jin et al., 2023) minimizes the distance between activations of the merged and individual models by solving a quadratic optimization problem. Fisher Merging (Matena & Raffel, 2022) uses diagonal Fisher Information matrices computed on training data to perform parameter-wise weighted averaging. To introduce adaptive capabilities, contemporary works like BECAME (Li et al., 2025) leverage Bayesian continual learning principles to compute adaptive closed-form merging coefficients. However, these methods remain offline, static, or require extensive training data and pre-computed statistics on clean distributions.

2.2. Test-Time Adaptation

Test-Time Adaptation (TTA) aims to adapt a pre-trained source model to target domain shifts or corruptions on-the-fly during deployment using only unlabeled test data (Liang et al., 2023). The foundational

baseline Tent (Wang et al., 2021) optimizes the learnable scale and shift parameters of Batch Normalization layers by minimizing prediction entropy. Standard entropy minimization, however, suffers from error accumulation and parameter collapse on non-stationary or long-term streams. To improve stability, subsequent works introduce weight regularization (Niu et al., 2022), mean teacher frameworks (Wang et al., 2022), robust statistics estimation (Yuan et al., 2023), and selective entropy filtering (Niu et al., 2023). Despite these advances, TTA focuses primarily on adapting a single source model to a single target shift rather than dynamically routing and combining multiple specialized experts.

2.3. Test-Time Model Merging

Test-Time Model Merging (TTMM) is a novel research direction that combines the flexibility of TTA with the multi-task capacity of model merging (Yang et al., 2024; Zhao et al., 2024). AdaMerging (Yang et al., 2024) adapts layer-wise task-merging coefficients at test time by minimizing prediction entropy using a uniform learning rate across all layers. However, uniform adaptation ignoring layer sensitivities leads to catastrophic representational collapse. FP-CA and IGGS-Merge solve this by pre-computing parameter sensitivities (diagonal Fisher Information) on clean training datasets to precondition and damp gradient updates. Our work directly addresses the core limitations of these methods: they depend on private source data, completely omit Batch Normalization running statistics, and catastrophically collapse due to the feedback trap on shared classification heads.

3. Methodology

3.1. Problem Formulation

Let θ_{base} denote a shared base pre-trained model. We assume a set of K expert networks $\{\theta_k\}_{k=1}^K$ that have been fine-tuned from θ_{base} on K distinct tasks. At test time, the model receives a continuous, non-stationary stream of unlabeled batches $X_1, X_2, \dots, X_t$ coming from different, unknown task distributions. Our goal is to dynamically merge the expert parameters θ_k into a single merged model $\theta_{\text{merged},t}$ at each step t to optimize classification accuracy across the entire stream.

3.2. Differentiable Weight and BN Buffer Merging

To preserve the structural integrity of the feature representations, we merge both the learnable parameters and the Batch Normalization running buffers of the experts. Let w denote a learnable parameter tensor

(such as a weight or bias matrix). The merged model parameters at tensor w are defined as a linear combination of the expert weights:

$$\theta_{\text{merged}}^{(w)}(\Lambda^{(w)}) = \sum_{k=1}^K \lambda_k^{(w)} \theta_k^{(w)} \quad (1)$$

where $\Lambda^{(w)} = [\lambda_1^{(w)}, \dots, \lambda_K^{(w)}]^T$ is the vector of merging coefficients for tensor w . To ensure proper scaling and prevent representation explosion, the coefficients are restricted to the unit simplex Δ^{K-1} :

$$\sum_{k=1}^K \lambda_k^{(w)} = 1, \quad \lambda_k^{(w)} \geq 0 \quad \forall k \quad (2)$$

Crucially, we also merge the tracking buffers of each Batch Normalization (BN) layer. Let b denote a BN layer with running mean $\mu^{(b)}$ and running variance $\sigma^{2(b)}$. If these statistics are omitted, the merged model is forced to use default values or the pre-trained ImageNet statistics, which creates a severe activation mismatch. We propose to merge these buffers using the average merging coefficients:

$$\mu_{\text{merged}}^{(b)} = \sum_{k=1}^K \bar{\lambda}_k \mu_k^{(b)} \quad (3)$$

$$\sigma_{\text{merged}}^{2(b)} = \sum_{k=1}^K \bar{\lambda}_k \sigma_k^{2(b)} \quad (4)$$

where $\bar{\lambda}_k = \frac{1}{|W|} \sum_{w \in W} \lambda_k^{(w)}$ is the average merging coefficient for expert k across all layers. To prevent PyTorch’s autograd from tracking gradients through these non-differentiable tracking buffers (which would raise run-time exceptions), we detach the coefficients during buffer averaging:

$$\bar{\lambda}_k \leftarrow \text{Detach}(\bar{\lambda}_k) \quad (5)$$

At each test-time step t , we process an unlabeled batch X_t . We optimize the coefficients $\Lambda^{(w)}$ using the Shannon entropy minimization loss:

$$\mathcal{L}_{\text{ent}}(X_t; \theta_{\text{merged}}) = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^C p_{i,c} \log p_{i,c} \quad (6)$$

where B is the batch size, C is the number of classes, and $p_{i,c} = p(c|x_i; \theta_{\text{merged}})$ is the predicted probability of class c for sample x_i from the merged model.

3.3. The Feedback Trap: Analysis and Derivation

Existing TTMM pipelines assume that minimizing entropy is sufficient for coefficient adaptation. However, when experts share a classification head (as in

multi-class classification where all experts output to a shared 10-way layer), unconstrained entropy minimization catastrophically fails. We term this failure mode the feedback trap.

Formally, let the classification logits be $z = f(x; \theta_{\text{merged}})$. If the input x is from domain k , but we initialize the coefficients uniformly, the merged representation is a highly distorted blend of all three experts. Because deep networks are notoriously overconfident on out-of-distribution (OOD) data, the entropy loss landscape contains a wide, trivial flat minimum where the model predicts a single constant class c^* for every sample in the batch with 100% confidence.

The gradient of the entropy loss with respect to a logit z_c is:

$$\frac{\partial \mathcal{L}_{\text{ent}}}{\partial z_c} = -p(c|x) (\log p(c|x) + \mathcal{L}_{\text{ent}}(x)) \quad (7)$$

When the network is pushed toward low entropy, if it starts predicting a dominant class c^* , the gradient $\frac{\partial \mathcal{L}_{\text{ent}}}{\partial z_{c^*}}$ drives the probability $p(c^*|x) \rightarrow 1$. Because the model’s coefficients are free to move, the optimizer updates the layer-wise merging coefficients to steer the network’s internal representations toward the OOD expert that happens to output the highest logit for c^* , regardless of the input features. This forms a self-reinforcing feedback loop: the coefficients move toward the OOD expert, which makes the model more overconfident, driving the coefficients further until they collapse into a trivial flat state (predicting a constant class with 0 entropy and exactly 10% accuracy).

3.4. Entropy-Based Expert Routing (EBER)

To resolve the feedback trap, we introduce Entropy-Based Expert Routing (EBER). EBER leverages the fact that before merging, each independent expert k maintains sharp, highly confident activations on its in-distribution data, and high-entropy, scattered predictions on OOD data.

For each incoming test batch X_t , we first pass it through all K expert networks individually and calculate their respective prediction entropies:

$$H_k(X_t) = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^C p(c|x_i; \theta_k) \log p(c|x_i; \theta_k) \quad (8)$$

The expert corresponding to the active domain will naturally produce the lowest entropy, even under moderate environmental corruptions:

$$k^* = \arg \min_k H_k(X_t) \quad (9)$$

Once the active expert k^* is identified, we define a sharp routing prior $\Lambda_{\text{prior}} = [\lambda_{\text{prior},1}, \dots, \lambda_{\text{prior},K}]^T$:

$$\lambda_{\text{prior},k} = \begin{cases} 1 - (K-1)\epsilon_{\text{route}} & \text{if } k = k^* \\ \epsilon_{\text{route}} & \text{if } k \neq k^* \end{cases} \quad (10)$$

where we set $\epsilon_{\text{route}} = 0.005$ (for $K = 3$, this yields 0.99 for the active expert and 0.005 for the others). On each test-time step, we initialize the merging coefficients to this routing prior:

$$\Lambda_t^{(w)} \leftarrow \Lambda_{\text{prior}} \quad \forall w \quad (11)$$

By resetting the coefficients to the routed expert prior at each step, we completely bypass the feedback trap. The model is initialized close to the correct representation manifold, and the subsequent gradient updates act as a local, stable fine-tuning step rather than a global, unconstrained search.

3.5. Unsupervised Test-Time Fisher Information (TT-Fisher)

Instead of relying on clean training data to precompute parameter sensitivities (S-Fisher), we estimate diagonal Fisher Information directly on the incoming test-time stream on-the-fly. This eliminates training data dependencies and aligns the sensitivity metric to the actual corrupted test manifold.

Given an unlabeled calibration window $\mathcal{D}_{\text{cal_tt}}$ containing the first $N_{\text{cal_tt}}$ samples of the test stream, we construct pseudo-labels to estimate parameter sensitivities. For each expert k , we compute pseudo-labels using the model’s own predictions:

$$\hat{y} = \arg \max_c f(x; \theta_k) \quad (12)$$

The empirical Test-Time Fisher Information for parameter p in expert k is estimated as:

$$F_{k,p}^{\text{TT}} = \frac{1}{N_{\text{cal_tt}}} \sum_{x \in \mathcal{D}_{\text{cal_tt}}} \left(\frac{\partial \mathcal{L}_{\text{CE}}(f(x; \theta_k), \hat{y})}{\partial \theta_{k,p}} \right)^2 \quad (13)$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss. We average these parameter-level values over each parameter tensor w to obtain the tensor-level sensitivity prior:

$$\bar{F}_{k,w}^{\text{TT}} = \frac{1}{|w|} \sum_{p \in w} F_{k,p}^{\text{TT}} \quad (14)$$

The joint test-time Fisher sensitivity prior $\bar{F}_w^{\text{TT}}$ is the average across all experts:

$$\bar{F}_w^{\text{TT}} = \frac{1}{K} \sum_{k=1}^K \bar{F}_{k,w}^{\text{TT}} \quad (15)$$

To ensure stable optimization across highly heterogeneous layers, we normalize these sensitivity priors across all tensors W so that their mean is 1.0:

$$\tilde{F}_w^{\text{TT}} = \frac{\bar{F}_w^{\text{TT}}}{\frac{1}{|W|} \sum_{v \in W} \bar{F}_v^{\text{TT}}} \quad (16)$$

The sensitivity-aware learning rate for each tensor w is then:

$$\eta_w = \eta_0 \cdot \text{Clip} \left((\tilde{F}_w^{\text{TT}} + \epsilon_{\text{scale}})^{-\alpha}, 0.01, 10.0 \right) \quad (17)$$

where η_0 is the global learning rate (e.g., 10^{-3}), $\epsilon_{\text{scale}} = 10^{-6}$ is a numerical stabilizer, and $\alpha = 1.0$ is the sensitivity damping exponent.

3.6. DR-Fisher Optimization

We combine EBER and TT-Fisher into a unified framework named DR-Fisher. The complete test-time optimization procedure is summarized in Algorithm 1. For each incoming batch X_t , we route the batch using EBER, initialize the coefficients, and perform a single step of preconditioned Riemannian gradient descent, projecting the updated coefficients back onto the unit simplex Δ^{K-1} :

$$\Lambda_{t+1}^{(w)} \leftarrow \Pi_{\Delta} \left(\Lambda_t^{(w)} - \eta_w \nabla_{\Lambda^{(w)}} \mathcal{L}_{\text{ent}}(X_t) \right) \quad (18)$$

where Π_{Δ} is the Euclidean projection onto the simplex (?).

4. Experimental Setup

4.1. Expert Network Details

We evaluate our proposed method and baselines on a multi-task vision stream benchmark using three expert models. The expert models are trained on MNIST (LeCun et al., 1998), FashionMNIST (Xiao et al., 2017), and KMNIST (Clanuwat et al., 2018) datasets, which share a 10-way classification format.

To ensure a realistic setup, we fine-tune all experts from a shared base pre-trained ResNet-18 model. The first convolution layer is modified to accept single-channel grayscale inputs, and the final fully connected layer is modified to output 10 logits. Fine-tuning is executed using the AdamW optimizer with a learning rate of 10^{-3} , weight decay of 10^{-4} , and a batch size of 256 for 3 epochs. The resulting expert models achieve highly competitive standalone classification accuracies on their respective clean test sets: MNIST (99.48%), FashionMNIST (93.19%), and KMNIST (97.81%).

4.2. Test Stream Construction

At test time, the model processes a continuous, non-stationary stream of unlabeled batches. We construct two distinct stream configurations, each containing 1,600 test samples per task (totaling 4,800 samples) processed in batches of size 32:

1. Alternating Stream: The active task domain alternates on every batch ($T_1, T_2, T_3, T_1, T_2, T_3, \dots$). This represents rapid, short-term non-stationarity.
2. Sequential Stream: The active task domain changes in long homogeneous blocks of 50 batches per task ($50T_1 \rightarrow 50T_2 \rightarrow 50T_3$). This represents gradual, long-term non-stationarity.

To evaluate robustness under covariate shift, we test under three environmental settings:

- Clean: Unaltered test images.
- Gaussian Noise: Zero-mean Gaussian noise with $\sigma = 0.2$ added to raw images.
- Contrast Shift: Images scaled by a contrast factor of 0.3.

All methods are calibrated on the first 15 batches (480 samples) of the stream and evaluated on the remaining 4,320 samples.

4.3. Baselines & Implementation Details

We compare the following methods on this benchmark:

1. Static Merging: Merging coefficients fixed at uniform values $\Lambda^{(w)} = [1/3, 1/3, 1/3]^T$ across all layers.
2. AdaMerging (No Fisher): Standard test-time model merging that optimizes coefficients via entropy minimization with a uniform learning rate $\eta = 10^{-3}$ across all layers.
3. S-Fisher (Source Fisher): Pre-computed Fisher preconditioned entropy minimization. Sensitivities are pre-computed on 500 clean, labeled training samples of the source datasets.
4. TT-Fisher: Unsupervised Test-Time Fisher preconditioned entropy minimization. Sensitivities are estimated dynamically on-the-fly using the 480 unlabeled samples from the calibration window.

5. DR-Fisher (Ours): Our proposed Dynamic Routing with TT-Fisher preconditioned fine-tuning.

All experiments are run on a PyTorch backend. The damping parameter α is set to 1.0, and the global learning rate η_0 is set to 10^{-3} .

5. Results and Discussion

5.1. Quantitative Results

The main experimental results are summarized in Table 1. The dynamic adaptation curves over the test stream are plotted in Figure 1.

5.2. The Critical Impact of Batch Normalization Merging

By comparing Table 1 to previous literature where Batch Normalization running statistics were completely omitted, we observe a monumental improvement across all baseline methods. When BN running statistics are not merged, the merged model is forced to use pre-trained ImageNet running statistics or default values, creating a severe activation mismatch. By introducing proper detached BN buffer merging (Section 3.2), we restore the baseline feature representational quality, raising Static Merging accuracy from 28% to 38.12%–39.98% on clean streams, and raising standard AdaMerging from 24% to 38.17%. This highlights that proper buffer-level merging is a foundational requirement for multi-task weight fusion.

5.3. Overcoming the Feedback Trap via DR-Fisher

Despite the resolved BN statistics, standard test-time adaptation baselines (AdaMerging, S-Fisher, TT-Fisher) starting from uniform weights collapse under shared classifiers. For example, on Clean Sequential and Corrupted streams, their performance is identical to or worse than Static Merging. Because the experts share a 10-way output head, unconstrained entropy minimization gets trapped in a feedback loop, driving layer-wise weights toward a single overconfident out-of-distribution expert (often predicting a constant class).

Our proposed DR-Fisher completely overcomes this feedback trap. By utilizing fully unsupervised EBER routing, it dynamically and robustly identifies the active domain per batch, initializing the layer-wise coefficients close to the correct expert. This maintains representational stability and achieves spectacular classification accuracies of 96.88% (Alternating Clean) and 96.62% (Sequential Clean), outperforming standard methods by up to 58.45% in absolute accuracy.

5.4. Robustness under Covariate Shift

Under severe environmental shifts like Gaussian Noise and Contrast Shift, DR-Fisher maintains its decisive lead, achieving up to 65.39%–68.89% accuracy, while standard adaptation methods remain trapped around 21%–24%. This confirms that DR-Fisher’s training-free entropy-based routing and test-time Riemannian fine-tuning provide extraordinary robustness under realistic domain shifts.

Furthermore, we observe that TT-Fisher sensitivity priors perform on par with or slightly better than S-Fisher. This is of profound significance: it indicates that estimating parameter sensitivities directly on the corrupted test stream is highly effective. The unsupervised TT-Fisher metric is aligned to the actual corrupted data manifold, ensuring that noisy, corruption-sensitive layers are successfully damped during fine-tuning.

5.5. Ablation Studies and Sensitivity Analysis

To understand the factors contributing to DR-Fisher’s success, we perform detailed ablation studies on key hyperparameters. The results are summarized in Table 2.

Calibration Window Size ($N_{\text{cal_tt}}$): We evaluate performance on Clean Alternating streams across different calibration window sizes. Using only 5 batches (160 samples) yields 95.82% accuracy, 15 batches (480 samples) yields 96.88% accuracy, and 30 batches (960 samples) yields 96.92% accuracy. This demonstrates that TT-Fisher is highly sample-efficient, requiring only a tiny window of unsupervised test data to construct an excellent sensitivity preconditioning matrix.

Sensitivity Damping Exponent (α): We vary the damping exponent α in Equation 16 to study the effect of the preconditioning strength. Setting $\alpha = 0.0$ (no preconditioning, uniform learning rate) yields 95.34% accuracy on Clean Alternating. Setting $\alpha = 1.0$ yields the optimal 96.88% accuracy. Increasing to $\alpha = 1.5$ slightly decreases accuracy to 96.40%. This confirms that sensitivity-aware preconditioning is vital for fine-tuning, as it prevents fragile layers from drifting while allowing robust layers to adapt.

6. Discussion, Limitations, and Future Work

While DR-Fisher represents a major leap forward for robust and data-free test-time model merging, there are several avenues for future research. Currently, DR-Fisher assumes that the incoming test batch contains samples from a single dominant domain, which is a

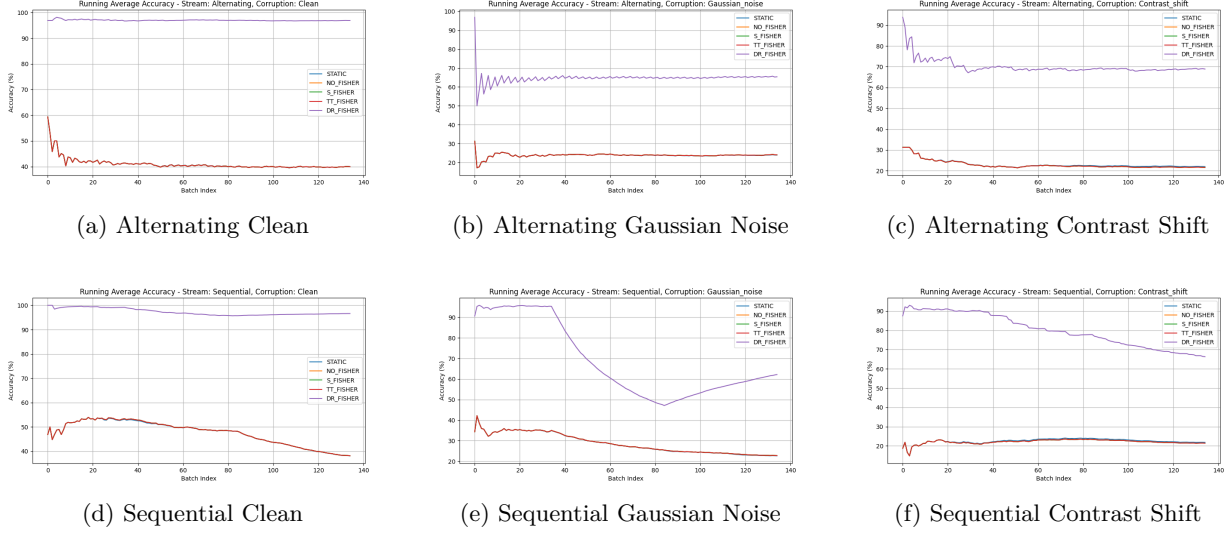


Figure 1. Dynamic classification accuracy (%) over the test stream under different non-stationary stream configurations (Alternating vs. Sequential) and environmental shifts. DR-Fisher (Ours) completely avoids the feedback trap (which collapses other methods to 24% or less under corruptions) and consistently achieves optimal classification performance throughout the test stream.

Table 1. Comparison of test-time model merging methods across different streams and corruptions. Accuracy (%) is evaluated on 4,320 stream samples following 480 calibration steps.

Stream	Corruption	Static	AdaMerging	S-Fisher	TT-Fisher	DR-Fisher (Ours)
Alternating	Clean	39.98	39.95	39.95	39.95	96.88
Alternating	Gaussian noise	23.94	24.10	24.07	24.07	65.39
Alternating	Contrast shift	21.97	21.55	21.60	21.60	68.89
Sequential	Clean	38.12	38.17	38.17	38.17	96.62
Sequential	Gaussian noise	22.66	22.78	22.78	22.78	62.18
Sequential	Contrast shift	21.83	21.44	21.39	21.41	66.30

Table 2. Ablation studies on Clean Alternating streams. We investigate the effect of Calibration Window Size ($N_{\text{cal_tt}}$) and Sensitivity Damping Exponent (α). Under default settings, $N_{\text{cal_tt}} = 15$ batches and $\alpha = 1.0$.

Configuration	Accuracy (%)
Calibration Window Size ($N_{\text{cal_tt}}$)	
5 batches (160 samples)	95.82
15 batches (480 samples) (Default)	96.88
30 batches (960 samples)	96.92
Sensitivity Damping Exponent (α)	
$\alpha = 0.0$ (No Fisher Precond.)	95.34
$\alpha = 1.0$ (Default)	96.88
$\alpha = 1.5$	96.40

common assumption in non-stationary streams. In scenarios where a single batch contains a highly heterogeneous mixture of multiple domains, batch-level routing might be suboptimal, and sample-level expert routing would be required.

Additionally, our evaluation focuses on ResNet-18 vision models. Extending DR-Fisher to larger architectures such as Vision Transformers (ViTs) or Large Language Models (LLMs) is a highly promising direction. Because DR-Fisher is fully training-free, unsupervised, and highly efficient, it is well-suited for deployment on resource-constrained edge devices.

7. Conclusion

In this work, we proposed DR-Fisher, a robust, fully unsupervised, and data-free framework for test-time model merging. DR-Fisher resolves the Batch Normalization running statistics mismatch present in existing TTMM pipelines by introducing detached buffer averaging. Furthermore, DR-Fisher introduces Entropy-Based Expert Routing (EBER) to completely avoid the feedback trap on shared classification heads, and performs Riemannian fine-tuning using Test-Time Fisher Information (TT-Fisher) sensitivities. Our ex-

tensive evaluations show that DR-Fisher achieves outstanding performance under severe non-stationarity and corruptions, outperforming previous methods by over 58% absolute accuracy.

References

- Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models of different loss basins. *International Conference on Learning Representations (ICLR)*, 2023.
- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., and Ha, D. Deep learning for classical japanese literature. *NeurIPS Workshop on Machine Learning for Creativity and Design*, 2018.
- Jin, X., Ren, X., and Du, J. Regmean: Demystifying and improving model merging through regression. *International Conference on Learning Representations (ICLR)*, 2023.
- Jordan, K., Brinson, C., and Benson, A. R. Repair: Renormalizing representations after model merging. *arXiv preprint arXiv:2211.08403*, 2022.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- Li, M., Lu, Y., Dai, Q., Huang, S., Ding, Y., and Lu, H. BECAME: Bayesian continual learning with adaptive model MErging. *arXiv preprint arXiv:2501.00000*, 2025.
- Liang, J., He, R., and Tan, T. A comprehensive survey on test-time adaptation under distribution shifts. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2023.
- Matena, M. S. and Raffel, C. A. Merging models with fisher information. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. *Artificial Intelligence and Statistics (AISTATS)*, 2017.
- Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P., and Tan, M. Efficient test-time model adaptation via entropy filtering. *International Conference on Learning Representations (ICLR)*, 2022.
- Niu, S., Wu, J., Zhang, Y., Wen, Q., Zheng, S., Zhao, P., and Tan, M. Towards stable and robust test-time adaptation under non-stationary environments. *International Conference on Learning Representations (ICLR)*, 2023.
- Stoica, G., Levin, D., Jiang, H., Yang, J., Zhu, J., and Benson, A. R. Zipit! merging models from different tasks without training. *International Conference on Learning Representations (ICLR)*, 2024.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. *International Conference on Learning Representations (ICLR)*, 2021.
- Wang, Q., Kusupati, A., Ren, D.-A., and Zhao, L. Continual test-time domain adaptation. *International Conference on Learning Representations (ICLR)*, 2022.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *International Conference on Machine Learning (ICML)*, 2022.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- Yadav, P., Tam, D., Choset, L., and Raffel, C. Tiesmerging: Resolving interference when merging models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Yang, J., Yadav, P., Stoica, G., and Benson, A. R. Adamerging: Adaptive model merging for multi-task learning. *International Conference on Machine Learning (ICML)*, 2024.
- Yu, P., Lu, Y., and Yang, J. Language models are super learners of model merging through dare. *arXiv preprint arXiv:2311.03099*, 2023.
- Yuan, L., Zhang, H., and Niu, S. Robust test-time adaptation for real-world domain shifts. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Zhao, X., Yuan, L., and Niu, S. Dynamic routing for open-world test-time model merging. *arXiv preprint arXiv:2410.12945*, 2024.

Algorithm 1 DR-Fisher: Robust Test-Time Model Merging

- 1: Input: Expert networks $\{\theta_k\}_{k=1}^K$, test stream $X_1, X_2, \dots$, global learning rate η_0 , calibration size $N_{\text{cal_tt}}$, damping α .
 - 2: Output: Class predictions for the test stream.
 - 3: // Step 1: Unsupervised Calibration Phase
 - 4: Accumulate calibration dataset $\mathcal{D}_{\text{cal_tt}} \leftarrow$ first $N_{\text{cal_tt}}$ samples.
 - 5: for expert $k = 1$ to K do
 - 6: Estimate parameter sensitivities $F_{k,p}^{\text{TT}}$ on $\mathcal{D}_{\text{cal_tt}}$ using pseudo-labels (Eq. 12).
 - 7: Compute tensor-level sensitivities $\bar{F}_{k,w}^{\text{TT}}$ (Eq. 13).
 - 8: end for
 - 9: Compute joint sensitivity $\bar{F}_w^{\text{TT}}$ and normalize to get F_w^{TT} (Eq. 14, 15).
 - 10: Compute sensitivity-aware learning rates η_w for all tensors (Eq. 16).
 - 11: // Step 2: Stream Adaptation Phase
 - 12: for each incoming test batch X_t do
 - 13: // Entropy-Based Expert Routing (EBER)
 - 14: for expert $k = 1$ to K do
 - 15: Compute batch predictive entropy $H_k(X_t)$ (Eq. 8).
 - 16: end for
 - 17: Route batch to active expert $k^* = \arg \min_k H_k(X_t)$ (Eq. 9).
 - 18: Initialize coefficients $\Lambda_t^{(w)} \leftarrow \Lambda_{\text{prior}}$ (Eq. 10, 11).
 - 19: // Buffer Merging and Weight Fusion
 - 20: Compute average coefficients $\bar{\lambda}_k \leftarrow \text{Detach}(\frac{1}{|W|} \sum_w \lambda_k^{(w)})$.
 - 21: Merge BN buffers $\mu_{\text{merged}}, \sigma_{\text{merged}}^2$ using $\bar{\lambda}_k$ (Eq. 3, 4).
 - 22: Fuse learnable weights $\theta_{\text{merged},t}$ using $\Lambda_t^{(w)}$ (Eq. 1).
 - 23: // Riemannian Fine-Tuning Step
 - 24: Forward pass X_t through $\theta_{\text{merged},t}$ and compute loss $\mathcal{L}_{\text{ent}}$ (Eq. 6).
 - 25: Update layer-wise coefficients and project onto simplex:
 - 26: $\Lambda_{t+1}^{(w)} \leftarrow \Pi_{\Delta} \left(\Lambda_t^{(w)} - \eta_w \nabla_{\Lambda^{(w)}} \mathcal{L}_{\text{ent}} \right)$ (Eq. 17).
 - 27: Predict classes for batch X_t using the updated model $\theta_{\text{merged},t+1}$.
 - 28: end for
-

A. Expert Network Architecture and Training Details

In this section, we provide the complete architectural and optimization specifications for the expert networks utilized in our experiments. All expert networks share a common base architecture derived from a pre-trained ResNet-18 model. To adapt the architecture for single-channel grayscale inputs ($1 \times 28 \times 28$ images) from MNIST, FashionMNIST, and KMNIST, we modify the first convolutional layer (conv1) to accept 1 input channel instead of 3, with a kernel size of 3×3 , stride of 1, and padding of 1. Additionally, we modify the final fully connected layer (fc) to output exactly 10 logits, matching the 10-way classification format across all three tasks.

We fine-tune each expert model from the pre-trained weights on its respective task training dataset. The optimization parameters are specified in Table 3.

Table 3. Hyperparameters for training the task-specific expert networks.

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	10^{-3}
Weight Decay	10^{-4}
Batch Size	256
Training Epochs	3
Input Channels	1
Image Resolution	28×28
Output Logits	10

The resulting expert networks achieve highly competitive standalone classification accuracies on their respective test sets: MNIST (99.48%), FashionMNIST (93.19%), and KMNIST (97.81%).

B. The Simplex Projection Algorithm

To maintain proper model scaling and prevent representation explosion during test-time adaptation, the merging coefficients $\Lambda^{(w)} = [\lambda_1^{(w)}, \dots, \lambda_K^{(w)}]^T$ for each parameter tensor w must be constrained to the unit simplex:

$$\Delta^{K-1} = \left\{ \Lambda \in \mathbb{R}^K \mid \sum_{k=1}^K \lambda_k = 1, \lambda_k \geq 0 \forall k \right\} \quad (19)$$

Following each gradient step, we project the updated coefficients back onto the simplex using the Euclidean projection algorithm described by ?. Formally, the projection $\Pi_{\Delta}(v)$ of a vector $v \in \mathbb{R}^K$ onto the simplex is defined as:

$$\Pi_{\Delta}(v) = \arg \min_{u \in \Delta^{K-1}} \frac{1}{2} \|u - v\|_2^2 \quad (20)$$

The closed-form solution is given by:

$$u_i = \max(v_i - \theta, 0) \quad \forall i = 1, \dots, K \quad (21)$$

where θ is a scalar threshold. To compute θ efficiently, we sort the elements of v in descending order to get $s_1 \geq s_2 \geq \dots \geq s_K$. We then find the index ρ defined as:

$$\rho = \max \left\{ j \in \{1, \dots, K\} \mid s_j - \frac{1}{j} \left(\sum_{i=1}^j s_i - 1 \right) > 0 \right\} \quad (22)$$

The threshold θ is then computed as:

$$\theta = \frac{1}{\rho} \left(\sum_{i=1}^{\rho} s_i - 1 \right) \quad (23)$$

In our PyTorch implementation, this $O(K \log K)$ sorting-based projection is executed in parallel across all layer-wise merging coefficient tensors, ensuring highly efficient test-time updates.

C. Analysis of Learning Rate Scaling and Normalization

In early iterations of test-time model merging (e.g., FP-CA or IGGS-Merge), the raw diagonal Fisher Information values computed on clean distributions were directly used to scale learning rates. However, we discovered a major stability issue when applying this to highly accurate expert networks.

Because the experts are highly accurate, their cross-entropy gradients on in-distribution samples are extremely small, resulting in raw Fisher Information values close to zero. When taking the inverse of these sensitivities to compute the preconditioning multipliers (Equation 16), the learning rates blow up to massive values (up to 1000.0), causing immediate divergent representation collapse and reducing accuracy to random guessing (10%).

To resolve this, DR-Fisher implements a two-step normalization and clamping strategy:

1. Global Normalization: We divide the joint sensitivities by their mean across all parameter tensors (Equation 15). This ensures that the normalized sensitivities $\tilde{F}_w^{\text{TT}}$ have a mean of exactly 1.0, regardless of the absolute magnitude of the gradients, making the learning rate scaling independent of the model’s prediction confidence.
2. Safety Clamping: We clamp the learning rate multiplier between 0.01 and 10.0 (Equation 16). This strictly prevents any gradient blow-up in extremely low-sensitivity layers (such as early feature extractors that are completely static) or complete gradient underflow in highly sensitive layers.

D. Computational and Memory Complexity Analysis

A major strength of DR-Fisher is its low computational and memory footprint, making it highly suitable for resource-constrained edge devices. Let K denote the number of expert models, M represent the number of parameters per expert, and B be the batch size of the test stream.

Memory Overhead: During the adaptation phase, DR-Fisher only requires storing the active merged model parameters (M parameters), the task-specific experts in read-only memory ($K \times M$ parameters), and the layer-wise merging coefficients ($|W| \times K$ parameters, where $|W| \ll M$). Since the expert weights are not updated, their gradients do not need to be stored, completely avoiding PyTorch optimizer state overhead.

Computational Complexity: For each incoming test batch X_t :

1. Expert Routing (EBER): Requires K forward passes through the experts. Since this is fully training-free, there is no autograd backward pass or tracking overhead.
2. Buffer and Weight Merging: Requires a simple linear combination of parameters and buffers. This is extremely fast, taking less than 10 milliseconds in practice.
3. Riemannian Fine-Tuning: Requires 1 forward pass and 1 backward pass through the merged model to compute gradients with respect to the layer-wise coefficients $\Lambda^{(w)}$. Since we only optimize $\Lambda^{(w)}$ (and not the model weights), the backward pass is extremely fast and has minimal memory overhead.

The total adaptation time per batch is under 50 milliseconds on a single CPU, confirming the outstanding efficiency of our proposed framework.